COMPLETE SYSTEM ARCHITECTURE DOCUMENT

AI-Based Road Hazard Detection System - 24 Hour Hackathon

---

1. SYSTEM OVERVIEW

Three-Component Architecture

```
┌─────────────────────────┐
│ Testing Dashboard       │ ← Where judges/users test the system
│ (Frontend)              │
└─────────────────────────┘
          │
          ↓ WebSocket (Real-time bidirectional)
          │
┌─────────────────────────┐
│ FastAPI Backend         │ ← Brain of the system (AI + Logic)
│ + AI Engine             │
└─────────────────────────┘
          │
          ↓ REST API + WebSocket
          │
┌─────────────────────────┐
│ Main Admin Panel        │ ← Command center for authorities
│ (Frontend)              │
└─────────────────────────┘
```

---

2. COMPONENT DETAILS

2.1 TESTING DASHBOARD (Frontend 1)

Technology Stack:

- React.js / Next.js

- WebSocket (Socket.io client)
- HTML5 Canvas API
- Tailwind CSS / Material-UI

Purpose:

Demo interface where users can upload videos/images and see AI predictions in real-time

Features:

A. Upload Interface

- **Video Upload**: Accepts .mp4, .avi, .mov files
- **Image Upload**: Accepts .jpg, .png files
- **Live Camera Feed** (Optional): For live demo

B. Real-Time Display Panel

- **Video Player**: Shows uploaded video playing
- **Annotation Overlay**: Displays bounding boxes, labels, severity scores
- **Frame Info**: Current timestamp, FPS counter

C. Detection Sidebar

- **Live Detection Feed**: Scrolling list of detections as they happen
    - Pothole detections (with severity badge)
    - Animal detections (with behavior status)
    - Accident alerts (with severity score)
- **Count Metrics**: Total detections by type
- **Filter Options**: Show only high-severity alerts

D. Upload Flow

```
User selects file
  ↓
File loaded in browser
  ↓
Video plays in <video> element
  ↓
Canvas extracts frames (every 3rd-5th frame)
  ↓
Frame converted to base64
  ↓
Sent to backend via WebSocket
  ↓
Receives annotated frame back
  ↓
Displays on screen
```

Data Flow (Testing Dashboard):

- **Outgoing**: Video frames (base64 JPEG, 50-80KB each)
- **Incoming**: Annotated frames (base64 JPEG with boxes drawn) + Detection JSON

2.2 FASTAPI BACKEND + AI ENGINE (Core System)

Technology Stack:

- Python 3.10+
- FastAPI (Web framework)
- WebSocket support (python-socketio)
- PyTorch / TensorFlow
- OpenCV (cv2)
- Ultralytics YOLOv8
- MongoDB / PostgreSQL (Database)
- NumPy, Pandas

Purpose:

Central processing unit - handles all AI inference, risk assessment, and data management

Architecture Layers:

```
┌─────────────────────────────────────────────────────┐
│      API LAYER (FastAPI)                            │
│  ├─ WebSocket endpoint (/ws)                        │
│  ├─ REST endpoints (/api/*)                         │
│  └─ CORS & Authentication                           │
└─────────────────────────────────────────────────────┘
                        │
                        ↓
┌─────────────────────────────────────────────────────┐
│     INPUT PROCESSING LAYER                          │
│  ├─ Frame decoder (base64 → NumPy)                  │
│  ├─ Image preprocessing                             │
│  └─ Night-time enhancement (optional)               │
└─────────────────────────────────────────────────────┘
                        │
                        ↓
┌─────────────────────────────────────────────────────┐
│       AI MODEL LAYER                                │
│  (3 Parallel Detection Modules)                     │
│                                                     │
│  ┌──────────────────────────────────────────┐       │
│  │  MODULE 1: Pothole Detection             │       │
│  ├──────────────────────────────────────────┤       │
│  │ • YOLOv8 model (pothole.pt)              │       │
```

```
│   │ • Detects potholes in frame   │      │
│   │ • Returns: bbox, confidence   │      │
│   └──────────────────────────────────────┘      │
│                                │
│   ┌──────────────────────────────────────┐      │
│   │ MODULE 2: Animal Detection    │      │
│   ├──────────────────────────────────────┤      │
│   │ • YOLOv8 model (animal.pt)    │      │
│   │ • Detects: dog, cow, goat,    │      │
│   │   pig, cat, etc.              │      │
│   │ • Returns: bbox, class, conf  │      │
│   └──────────────────────────────────────┘      │
│                                │
│   ┌──────────────────────────────────────┐      │
│   │ MODULE 3: Accident Detection  │      │
│   ├──────────────────────────────────────┤      │
│   │ • Vehicle detection (YOLO)    │      │
│   │ • Motion analysis (frame diff)│      │
│   │ • Collision detection logic   │      │
│   │ • Returns: accident flag,     │      │
│   │   severity score              │      │
│   └──────────────────────────────────────┘      │
└──────────────────────────────────────────────────┘
                  │
                  ↓
┌──────────────────────────────────────────────────┐
│   FEATURE EXTRACTION LAYER            │
│  (Implements USP Sub-Features)        │
│                                │
│  POTHOLE FEATURES:                    │
│  ├─ Severity Classification           │
│  │   └─ Logic: size, shape analysis   │
│  │   └─ Output: Low/Medium/High       │
│  ├─ Context-Aware Risk Adjustment     │
│  │   └─ Consider: road position, traffic│
│  └─ Geo-Tagging                       │
│      └─ Attach: Camera ID, coordinates │
│                                │
│  ANIMAL FEATURES:                     │
│  ├─ Behavior Classification           │
│  │   └─ Analyze: pose, movement       │
│  │   └─ Categories: sitting, standing, │
│  │      walking, running, crossing    │
│  ├─ Trajectory Prediction             │
│  │   └─ Track across frames           │
│  │   └─ Predict road entry            │
│  ├─ Region-Based Risk                 │
│  │   └─ Define: road zone vs safe zone │
│  │   └─ Alert if in road zone         │
```

```
│   └─ Night-Time Optimization              │
│       └─ Low-light preprocessing          │
│                                           │
│   ACCIDENT FEATURES:                      │
│   ├─ Severity Scoring (0-100)             │
│   │   └─ Factors: speed, vehicle count,   │
│   │      vehicle type, human presence     │
│   ├─ Hit-and-Run Detection                │
│   │   └─ Track post-collision movement    │
│   ├─ Traffic Disruption Analysis          │
│   │   └─ Compare pre/post density         │
│   └─ Predictive Accident Detection        │
│       └─ Sudden stops, abnormal paths     │
└───────────────────────────────────────────┘
                    │
                    ↓
┌───────────────────────────────────────────┐
│   MULTI-HAZARD FUSION LAYER               │
│   (Cross-Module Intelligence)             │
│                                           │
│   ├─ Combined Risk Assessment             │
│   │   Example: Animal near pothole        │
│   │            + Fast vehicle             │
│   │            = CRITICAL RISK            │
│   │                                       │
│   ├─ Unified Risk Scoring                 │
│   │   └─ Normalize all risks to 0-100     │
│   │                                       │
│   └─ Priority Ranking                     │
│       └─ Determine which alerts sent      │
│          to Admin Panel                   │
└───────────────────────────────────────────┘
                    │
                    ↓
┌───────────────────────────────────────────┐
│   ANNOTATION LAYER                        │
│   (Visual Output Generation)              │
│                                           │
│   ├─ Draw Bounding Boxes (OpenCV)         │
│   │   └─ Color-coded by severity:         │
│   │       • Green = Low                   │
│   │       • Orange = Medium               │
│   │       • Red = High                    │
│   │                                       │
│   ├─ Add Text Labels                      │
│   │   └─ Format: "Type - Severity"        │
│   │   └─ Example: "Pothole - HIGH"        │
│   │                                       │
│   ├─ Draw Confidence Scores               │
│   │   └─ Show % next to detection         │
```

```
│  │                                │
│  └─ Add Severity Badges            │
│     └─ Visual icons for quick ID   │
└────────────────────────────────────┘
              │
              ↓
┌────────────────────────────────────┐
│    ALERT GENERATION LAYER          │
│                                    │
│  Decision Logic:                   │
│  IF severity >= 70 OR risk = CRITICAL │
│  THEN:                             │
│   ├─ Create Alert Object           │
│   │  └─ {id, type, severity,       │
│   │      timestamp, location,      │
│   │      image, metadata}          │
│   ├─ Save to Database              │
│   ├─ Send to Admin Panel (WebSocket) │
│   └─ Log for analytics             │
└────────────────────────────────────┘
              │
              ↓
┌────────────────────────────────────┐
│    OUTPUT LAYER                    │
│                                    │
│  To Testing Dashboard:             │
│  ├─ Annotated frame (base64 with boxes) │
│  └─ Detection JSON array           │
│                                    │
│  To Admin Panel:                   │
│  ├─ Alert events (WebSocket)       │
│  └─ Analytics data (REST API)      │
│                                    │
│  To Database:                      │
│  ├─ All detections (for history)   │
│  ├─ Alert records                  │
│  └─ Analytics aggregates           │
└────────────────────────────────────┘
```

Database Schema:

```
COLLECTIONS / TABLES:

1. detections
   ├─ id (UUID)
   ├─ type (pothole/animal/accident)
   ├─ severity (low/medium/high)
   ├─ confidence (0-1)
   ├─ bbox [x, y, w, h]
   ├─ timestamp (ISO datetime)
```

```
   ├─ camera_id (string)
   ├─ metadata (JSON - specific to type)
   └─ image_thumbnail (base64 or URL)

2. alerts
   ├─ id (UUID)
   ├─ detection_id (foreign key)
   ├─ severity_score (0-100)
   ├─ alert_level (medium/high/critical)
   ├─ status (new/acknowledged/resolved)
   ├─ timestamp
   └─ location_data (lat, long, camera_id)

3. analytics_cache
   ├─ metric_name (string)
   ├─ value (number/JSON)
   ├─ time_bucket (hourly/daily)
   └─ last_updated
```

API Endpoints:

```
WebSocket:
  /ws
    - Bidirectional real-time communication
    - Events:
      • process_frame (receive from Testing Dashboard)
      • annotated_frame (send to Testing Dashboard)
      • new_alert (broadcast to Admin Panel)

REST API:
  POST /api/upload
    - Alternative to WebSocket for single image

  GET /api/alerts
    - Query parameters: severity, type, start_date, end_date
    - Returns: List of alerts

  GET /api/analytics/summary
    - Returns: Overall statistics
    - Response: {
        total_detections,
        by_type {pothole, animal, accident},
        by_severity {low, medium, high},
        today_count,
        hourly_breakdown
      }

  GET /api/analytics/heatmap
```

```
  - Returns: Location-based frequency data

GET /api/detections/:id
  - Get specific detection details

PATCH /api/alerts/:id
  - Update alert status (acknowledge/resolve)
```

2.3 MAIN ADMIN PANEL (Frontend 2)

Technology Stack:

- React.js / Next.js
- WebSocket (Socket.io client)
- Chart.js / Recharts (for analytics)
- Leaflet / Mapbox (for heatmap)
- Tailwind CSS / Material-UI

Purpose:

Command center for traffic authorities to monitor alerts and view analytics

Features:

A. Real-Time Alert Feed

- **Alert Cards**: Display new alerts as they arrive
    - Type icon (pothole/animal/accident)
    - Severity badge (color-coded)
    - Timestamp
    - Location (Camera ID)
    - Thumbnail image
    - Action buttons (Acknowledge, View Details)
- **Sorting/Filtering**:
    - By severity (High → Low)
    - By type
    - By time (Latest first)
    - By status (New/Acknowledged/Resolved)

B. Analytics Dashboard

Metric Cards:

- Total Detections (today)
- Active Alerts (unresolved)
- Potholes Detected
- Animals Detected

- Accidents Detected
- Average Response Time

Charts:

- **Line Chart**: Incidents over time (hourly/daily)
- **Bar Chart**: Detections by type
- **Pie Chart**: Severity distribution
- **Area Chart**: Traffic disruption trends

Heatmap:

- **Map View**: Shows camera locations
- **Color Intensity**: Frequency of detections
- **Hotspot Markers**: Areas with most animal activity
- **Click Details**: View location-specific history

C. Historical Log

- **Table View**: All past detections
- **Search**: By date, type, severity
- **Export**: Download as CSV/PDF
- **Details Modal**: Click to see full detection info

D. Settings Panel (Optional)

- Alert threshold configuration
- Camera management
- User preferences

Data Flow (Admin Panel):

- **Real-time (WebSocket)**: New alert notifications
- **Periodic (REST API)**: Analytics data (every 30 seconds)
- **On-demand (REST API)**: Historical queries, filters

---

3. COMPLETE DATA FLOW

3.1 PRIMARY FLOW (Real-Time Detection)

```
┌──────────────────────────────────────────────────────────────┐
│                          USER ACTION                         │
│  User uploads video in Testing Dashboard                     │
└──────────────────────────────────────────────────────────────┘
```

```
                    ↓
┌──────────────
```

```
│  Step 11: Multi-Hazard Fusion                          │
│         Combine risks across modules                   │
│         Example: Animal + Pothole + Fast Vehicle       │
│               = CRITICAL combined risk                 │
│                      ↓                                 │
│  Step 12: Visual Annotation                            │
│         annotated_frame = frame.copy()                 │
│         For each detection:                            │
│         ├─ cv2.rectangle() - Draw box                  │
│         ├─ cv2.putText() - Add label                   │
│         └─ Color by severity (Red/Orange/Green)        │
│                      ↓                                 │
│  Step 13: Alert Decision                               │
│         IF severity >= 70 OR risk = CRITICAL:          │
│         ├─ Create alert object                         │
│         ├─ Save to database                            │
│         └─ Queue for Admin Panel                       │
│                      ↓                                 │
│  Step 14: Encode output                                │
│         annotated_base64 = encode(annotated_frame)     │
│                      ↓                                 │
│  Step 15: Send response via WebSocket                  │
│         socket.emit('annotated_frame', {               │
│           annotatedFrame: base64,                      │
│           detections: [...],                           │
│           timestamp: 1.5                               │
│         })                                             │
└────────────────────────────────────────────────────────┘
                      ↓
                (Network: 15ms)
                      ↓
┌────────────────────────────────────────────────────────┐
│           TESTING DASHBOARD (Browser)                  │
│                                                        │
│  Step 16: Receive annotated frame                      │
│  Step 17: Display frame                                │
│         <img src={data.annotatedFrame} />              │
│         → User sees video with bounding boxes          │
│                                                        │
│  Step 18: Update detection sidebar                     │
│         Map detections to UI cards                     │
│         Show: Type, Severity, Confidence               │
│                                                        │
│  Step 19: Check for critical alerts                    │
│         IF detections include high severity:           │
│         └─ Trigger sound/visual notification           │
```

```
│                                                                │
│  Step 20: Extract next frame                                   │
│          → Loop back to Step 3                                 │
└────────────────────────────────────────────────────────────────┘
```

3.2 SECONDARY FLOW (Alert to Admin Panel)

```
┌────────────────────────────────────────────────────────────────┐
│            FASTAPI BACKEND                                     │
│  Critical detection identified (Step 13 above)                 │
└────────────────────────────────────────────────────────────────┘
                    ↓
┌────────────────────────────────────────────────────────────────┐
│          ALERT PROCESSING                                      │
│                                                                │
│  Step A: Create Alert Record                                   │
│         alert = {                                              │
│           id: UUID,                                            │
│           type: "pothole",                                     │
│           severity: "high",                                    │
│           severity_score: 85,                                  │
│           timestamp: ISO datetime,                             │
│           location: "Camera-5",                                │
│           camera_coords: [lat, lng],                           │
│           detection_data: {...},                               │
│           image_thumbnail: base64_crop,                        │
│           status: "new"                                        │
│         }                                                      │
│                    ↓                                           │
│  Step B: Save to Database                                      │
│         db.alerts.insert_one(alert)                            │
│         db.detections.insert_one(detection)                    │
│                    ↓                                           │
│  Step C: Broadcast via WebSocket                               │
│         admin_socket.emit('new_alert', alert)                  │
│                    ↓                                           │
│  Step D: Update Analytics Cache                                │
│         Increment counters:                                    │
│         ├─ total_alerts_today += 1                             │
│         ├─ pothole_count += 1                                  │
│         └─ high_severity_count += 1                            │
└────────────────────────────────────────────────────────────────┘
                    ↓
```

```
┌──────────────
```

```
│          SELECT COUNT(*) FROM detections                    │
│          WHERE timestamp >= TODAY                           │
│                                                             │
│  Query 5: Hourly Breakdown                                  │
│          SELECT HOUR(timestamp), COUNT(*)                   │
│          FROM detections                                    │
│          WHERE timestamp >= TODAY                           │
│          GROUP BY HOUR(timestamp)                           │
│                                                             │
│  Aggregate Results into JSON                                │
│          {                                                  │
│            total_detections: 1247,                          │
│            by_type: {pothole: 523, animal: 412, ...},       │
│            by_severity: {high: 89, medium: 234, ...},       │
│            today: 47,                                       │
│            hourly: [{hour: 0, count: 3}, ...]               │
│          }                                                  │
└─────────────────────────────────────────────────────────────┘

                    ↓
                Return JSON
                    ↓
┌─────────────────────────────────────────────────────────────┐
│          MAIN ADMIN PANEL (Browser)                         │
│                                                             │
│  Receive analytics data                                     │
│  Update UI components:                                      │
│  ├─ Metric cards (numbers)                                  │
│  ├─ Line chart (hourly data)                                │
│  ├─ Bar chart (by type)                                     │
│  └─ Pie chart (severity distribution)                       │
└─────────────────────────────────────────────────────────────┘
```

---

4. QUICK REFERENCE Q&A

Q1: Does it send whole video or single frames?

A: Single frames, one at a time. Each frame is ~80KB.

Q2: Does backend return video or data?

A: Both - An annotated image (with boxes already drawn) + JSON detection data.

Q3: Does it send each frame or collectively?

A: Each frame individually in real-time loop.

Q4: Are boxes drawn in backend or frontend?

A: Backend draws boxes using OpenCV (recommended). Frontend just displays the image.

Q5: How fast is it?

A: 100-200ms per frame = 5-10 FPS output (smooth for demo).

Q6: Why not use Spring Boot?

A: Python FastAPI is faster to build, natively supports ML libraries, and handles WebSockets easily.

Q7: Where is the database?

A: Connected to FastAPI backend. Stores all detections, alerts, and analytics.

Q8: Can we demo without internet?

A: Yes - Run everything locally (localhost). No external dependencies needed.

Q9: What if models are slow?

A: Skip frames (process every 3rd-5th frame), resize images to 640x480, use YOLOv8-nano.

Q10: How to showcase all features in demo?

A: Prepare 3 test videos:

- Video 1: Multiple potholes (shows severity classification)
- Video 2: Stray animals crossing (shows behavior detection)
- Video 3: Near-miss accident (shows accident detection + severity scoring)

---

5. IMPLEMENTATION PRIORITY (24-Hour Timeline)

HOUR 0-6: CORE FOUNDATION (MUST HAVE)

Backend:

- ✅ FastAPI setup with WebSocket
- ✅ Basic frame processing pipeline
- ✅ Load pre-trained YOLO models (pothole, animal)
- ✅ Simple bounding box drawing

Testing Dashboard:

- ✅ Video upload interface
- ✅ WebSocket connection
- ✅ Display annotated frames

Goal: End-to-end flow working (even if basic)

HOUR 6-12: AI FEATURES (CORE USPs)

Backend:

- ✅ Pothole severity classification
- ✅ Animal behavior detection (basic)
- ✅ Accident detection (motion-based)
- ✅ Database integration

Testing Dashboard:

- ✅ Detection sidebar with live updates
- ✅ Severity badges/colors

Admin Panel:

- ✅ Basic alert feed
- ✅ WebSocket connection for alerts

Goal: All 3 detection modules working + alerts flowing

HOUR 12-18: POLISH & ANALYTICS (IMPRESSIVE)

Backend:

- ✅ Multi-hazard fusion logic
- ✅ Analytics endpoints
- ✅ Alert generation rules

Admin Panel:

- ✅ Analytics dashboard
- ✅ Charts (line, bar, pie)
- ✅ Metric cards

Testing Dashboard:

- ✅ Better UI/UX
- ✅ Loading states

Goal: Complete system with analytics

HOUR 18-22: ADVANCED FEATURES (BONUS)

If time permits:

- ✅ Heatmap visualization
- ✅ Night-time detection demo
- ✅ Hit-and-run detection logic
- ✅ Trajectory prediction (animals)

Goal: Showcase unique USPs

HOUR 22-24: TESTING & PRESENTATION

Tasks:

- ✅ End-to-end testing
- ✅ Prepare demo videos (pre-tested)

- ✅ Create presentation slides
- ✅ Setup backup (if live demo fails)
- ✅ Document setup instructions

---

6. DEPLOYMENT ARCHITECTURE

Local Development (Recommended for Hackathon)

```
Your Laptop:
├─ Port 8000: FastAPI Backend
├─ Port 3000: Testing Dashboard
├─ Port 3001: Admin Panel
└─ Port 27017: MongoDB (or PostgreSQL 5432)

Network:
└─ All communicate via localhost (no internet needed)
```

For Remote Judging (Optional)

```
Backend:
├─ Deploy to: Render / Railway / Heroku
└─ URL: https://your-backend.onrender.com

Frontends:
├─ Deploy to: Vercel / Netlify
├─ Testing Dashboard: https://testing.vercel.app
└─ Admin Panel: https://admin.vercel.app

Database:
└─ MongoDB Atlas (free tier) / Supabase
```

---

7. TECHNOLOGY JUSTIFICATION

Component	Choice	Why
Backend Framework	FastAPI	Fast, async, WebSocket support, Python-native
AI Models	YOLOv8	Pre-trained, fast inference, easy fine-tuning

Frontend	React	Component reusability, fast development
WebSocket	Socket.io	Real-time bidirectional, easy to use
Database	MongoDB	Flexible schema, fast writes, easy setup
Image Processing	OpenCV	Industry standard, Python integration
Charts	Chart.js	Simple, lightweight, good visuals

---

8. FILE STRUCTURE

```
project-root/
│
├── backend/                # FastAPI Backend
│   ├── main.py             # FastAPI app entry point
│   ├── models/             # AI model files
│   │   ├── pothole.pt
│   │   ├── animal.pt
│   │   └── vehicle.pt
│   ├── services/
│   │   ├── ai_engine.py      # AI inference logic
│   │   ├── feature_extraction.py
│   │   └── alert_manager.py
│   ├── api/
│   │   ├── websocket.py      # WebSocket handlers
│   │   └── routes.py         # REST API endpoints
│   ├── database/
│   │   ├── models.py         # DB schemas
│   │   └── connection.py
│   └── requirements.txt
│
├── testing-dashboard/        # React Frontend 1
│   ├── src/
│   │   ├── components/
│   │   │   ├── VideoUploader.jsx
│   │   │   ├── VideoPlayer.jsx
│   │   │   └── DetectionSidebar.jsx
│   │   ├── services/
│   │   │   └── websocket.js
│   │   └── App.jsx
│   └── package.json
```

```
│
├── admin-panel/            # React Frontend 2
│   ├── src/
│   │   ├── components/
│   │   │   ├── AlertFeed.jsx
│   │   │   ├── Analytics.jsx
│   │   │   └── Heatmap.jsx
│   │   ├── services/
│   │   │   ├── api.js
│   │   │   └── websocket.js
│   │   └── App.jsx
│   └── package.json
│
└── README.md
```

---

9. SUCCESS METRICS (What Judges Will See)

Demo Flow:

1. **Upload test video** → System processes in real-time
2. **Bounding boxes appear** → Shows AI is working
3. **Alerts pop up in Admin Panel** → Shows integration
4. **Analytics update** → Shows data intelligence

Wow Factors:

- ✅ Real-time processing feels instant
- ✅ Color-coded severity (visual impact)
- ✅ Multiple hazard types detected simultaneously
- ✅ Professional dashboard UI
- ✅ Live analytics charts

Technical Highlights to Mention:

- ✅ Multi-modal AI (3 detection types)
- ✅ Behavior-based intelligence (not just object detection)
- ✅ Scalable architecture (ready for real CCTVs)
- ✅ Context-aware risk assessment (fusion logic)

---

10. RISK MITIGATION

Backup Plans:

If AI is too slow:

- Use smaller models (YOLOv8-nano)
- Process fewer frames (every 5th frame)
- Pre-process demo videos

If WebSocket fails:

- Fall back to REST API polling
- Use pre-recorded results

If database is down:

- Use in-memory storage (Python dict)
- Show cached analytics

If live demo crashes:

- Have recorded demo video ready
- Explain architecture from slides

COMPLETE SYSTEM ARCHITECTURE DOCUMENT

AI-Based Road Hazard Detection System - 24 Hour Hackathon

---

1. SYSTEM OVERVIEW

Three-Component Architecture

```
┌──────────────────────┐
│ Testing Dashboard    │ ← Where judges/users test the system
│ (Frontend)           │
└──────────────────────┘
```

```
        │
        ↓ WebSocket (Real-time bidirectional)
        │
┌──────────────────────────┐
│ FastAPI Backend          │ ← Brain of the system (AI + Logic)
│ + AI Engine              │
└──────────────────────────┘
        │
        ↓ REST API + WebSocket
        │
┌──────────────────────────┐
│ Main Admin Panel         │ ← Command center for authorities
│ (Frontend)               │
└──────────────────────────┘
```

---

2. COMPONENT DETAILS

2.1 TESTING DASHBOARD (Frontend 1)

Technology Stack:

- React.js / Next.js
- WebSocket (Socket.io client)
- HTML5 Canvas API
- Tailwind CSS / Material-UI

Purpose:

Demo interface where users can upload videos/images and see AI predictions in real-time

Features:

A. Upload Interface

- **Video Upload**: Accepts .mp4, .avi, .mov files
- **Image Upload**: Accepts .jpg, .png files
- **Live Camera Feed** (Optional): For live demo

B. Real-Time Display Panel

- **Video Player**: Shows uploaded video playing
- **Annotation Overlay**: Displays bounding boxes, labels, severity scores
- **Frame Info**: Current timestamp, FPS counter

C. Detection Sidebar

- **Live Detection Feed**: Scrolling list of detections as they happen
    - Pothole detections (with severity badge)
    - Animal detections (with behavior status)
    - Accident alerts (with severity score)
- **Count Metrics**: Total detections by type
- **Filter Options**: Show only high-severity alerts

D. Upload Flow

```
User selects file
    ↓
File loaded in browser
    ↓
Video plays in <video> element
    ↓
Canvas extracts frames (every 3rd-5th frame)
    ↓
Frame converted to base64
    ↓
Sent to backend via WebSocket
    ↓
Receives annotated frame back
    ↓
Displays on screen
```

Data Flow (Testing Dashboard):

- **Outgoing**: Video frames (base64 JPEG, 50-80KB each)
- **Incoming**: Annotated frames (base64 JPEG with boxes drawn) + Detection JSON

2.2 FASTAPI BACKEND + AI ENGINE (Core System)

Technology Stack:

- Python 3.10+
- FastAPI (Web framework)
- WebSocket support (python-socketio)
- PyTorch / TensorFlow
- OpenCV (cv2)
- Ultralytics YOLOv8
- MongoDB / PostgreSQL (Database)
- NumPy, Pandas

Purpose:

Central processing unit - handles all AI inference, risk assessment, and data management

Architecture Layers:

```
┌─────────────────────────────────────────────────────────┐
│  API LAYER (FastAPI)                                    │
│  ├─ WebSocket endpoint (/ws)                            │
│  ├─ REST endpoints (/api/*)                             │
│  └─ CORS & Authentication                               │
└─────────────────────────────────────────────────────────┘
                          │
                          ↓
┌─────────────────────────────────────────────────────────┐
│  INPUT PROCESSING LAYER                                 │
│  ├─ Frame decoder (base64 → NumPy)                      │
│  ├─ Image preprocessing                                 │
│  └─ Night-time enhancement (optional)                   │
└─────────────────────────────────────────────────────────┘
                          │
                          ↓
┌─────────────────────────────────────────────────────────┐
│  AI MODEL LAYER                                         │
│  (3 Parallel Detection Modules)                         │
│                                                         │
│  ┌──────────────────────────────────────────────┐       │
│  │ MODULE 1: Pothole Detection                  │       │
│  ├──────────────────────────────────────────────┤       │
│  │ • YOLOv8 model (pothole.pt)                  │       │
│  │ • Detects potholes in frame                  │       │
│  │ • Returns: bbox, confidence                  │       │
│  └──────────────────────────────────────────────┘       │
│                                                         │
│  ┌──────────────────────────────────────────────┐       │
│  │ MODULE 2: Animal Detection                   │       │
│  ├──────────────────────────────────────────────┤       │
│  │ • YOLOv8 model (animal.pt)                   │       │
│  │ • Detects: dog, cow, goat,                   │       │
│  │   pig, cat, etc.                             │       │
│  │ • Returns: bbox, class, conf                 │       │
│  └──────────────────────────────────────────────┘       │
│                                                         │
│  ┌──────────────────────────────────────────────┐       │
│  │ MODULE 3: Accident Detection                 │       │
│  ├──────────────────────────────────────────────┤       │
│  │ • Vehicle detection (YOLO)                   │       │
│  │ • Motion analysis (frame diff)               │       │
│  │ • Collision detection logic                  │       │
│  │ • Returns: accident flag,                    │       │
│  │   severity score                             │       │
│  └──────────────────────────────────────────────┘       │
└─────────────────────────────────────────────────────────┘
                          │
                          ↓
```

```
┌──────────────────────────────────────────────┐
│  FEATURE EXTRACTION LAYER                    │
│ (Implements USP Sub-Features)                │
│                                              │
│ POTHOLE FEATURES:                            │
│ ├─ Severity Classification                   │
│ │   └─ Logic: size, shape analysis           │
│ │   └─ Output: Low/Medium/High               │
│ ├─ Context-Aware Risk Adjustment             │
│ │   └─ Consider: road position, traffic      │
│ └─ Geo-Tagging                               │
│     └─ Attach: Camera ID, coordinates        │
│                                              │
│ ANIMAL FEATURES:                             │
│ ├─ Behavior Classification                   │
│ │   └─ Analyze: pose, movement               │
│ │   └─ Categories: sitting, standing,        │
│ │      walking, running, crossing            │
│ ├─ Trajectory Prediction                     │
│ │   └─ Track across frames                   │
│ │   └─ Predict road entry                    │
│ ├─ Region-Based Risk                         │
│ │   └─ Define: road zone vs safe zone        │
│ │   └─ Alert if in road zone                 │
│ └─ Night-Time Optimization                   │
│     └─ Low-light preprocessing               │
│                                              │
│ ACCIDENT FEATURES:                           │
│ ├─ Severity Scoring (0-100)                  │
│ │   └─ Factors: speed, vehicle count,        │
│ │      vehicle type, human presence          │
│ ├─ Hit-and-Run Detection                     │
│ │   └─ Track post-collision movement         │
│ ├─ Traffic Disruption Analysis               │
│ │   └─ Compare pre/post density              │
│ └─ Predictive Accident Detection             │
│     └─ Sudden stops, abnormal paths          │
└──────────────────────────────────────────────┘
                    │
                    ↓
┌──────────────────────────────────────────────┐
│  MULTI-HAZARD FUSION LAYER                   │
│ (Cross-Module Intelligence)                  │
│                                              │
│ ├─ Combined Risk Assessment                  │
│ │   Example: Animal near pothole             │
│ │            + Fast vehicle                  │
│ │            = CRITICAL RISK                 │
│ │                                            │
```

```
│  ├─ Unified Risk Scoring              │
│  │   └─ Normalize all risks to 0-100   │
│  │                                     │
│  └─ Priority Ranking                   │
│      └─ Determine which alerts sent    │
│         to Admin Panel                 │
└────────────────────────────────────────┘
                  │
                  ↓
┌────────────────────────────────────────┐
│    ANNOTATION LAYER                    │
│  (Visual Output Generation)            │
│                                        │
│  ├─ Draw Bounding Boxes (OpenCV)       │
│  │   └─ Color-coded by severity:       │
│  │       • Green = Low                 │
│  │       • Orange = Medium             │
│  │       • Red = High                  │
│  │                                     │
│  ├─ Add Text Labels                    │
│  │   └─ Format: "Type - Severity"      │
│  │   └─ Example: "Pothole - HIGH"      │
│  │                                     │
│  ├─ Draw Confidence Scores             │
│  │   └─ Show % next to detection       │
│  │                                     │
│  └─ Add Severity Badges                │
│      └─ Visual icons for quick ID      │
└────────────────────────────────────────┘
                  │
                  ↓
┌────────────────────────────────────────┐
│    ALERT GENERATION LAYER              │
│                                        │
│  Decision Logic:                       │
│  IF severity >= 70 OR risk = CRITICAL  │
│  THEN:                                 │
│   ├─ Create Alert Object               │
│   │   └─ {id, type, severity,          │
│   │       timestamp, location,         │
│   │       image, metadata}             │
│   ├─ Save to Database                  │
│   ├─ Send to Admin Panel (WebSocket)   │
│   └─ Log for analytics                 │
└────────────────────────────────────────┘
                  │
                  ↓
┌────────────────────────────────────────┐
│    OUTPUT LAYER                        │
│                                        │
│  To Testing Dashboard:                 │
```

```
│   ├─ Annotated frame (base64 with boxes) │
│   └─ Detection JSON array               │
│                                         │
│   To Admin Panel:                       │
│   ├─ Alert events (WebSocket)           │
│   └─ Analytics data (REST API)          │
│                                         │
│   To Database:                          │
│   ├─ All detections (for history)       │
│   ├─ Alert records                      │
│   └─ Analytics aggregates               │
└─────────────────────────────────────────┘
```

Database Schema:

```
COLLECTIONS / TABLES:

1. detections
   ├─ id (UUID)
   ├─ type (pothole/animal/accident)
   ├─ severity (low/medium/high)
   ├─ confidence (0-1)
   ├─ bbox [x, y, w, h]
   ├─ timestamp (ISO datetime)
   ├─ camera_id (string)
   ├─ metadata (JSON - specific to type)
   └─ image_thumbnail (base64 or URL)

2. alerts
   ├─ id (UUID)
   ├─ detection_id (foreign key)
   ├─ severity_score (0-100)
   ├─ alert_level (medium/high/critical)
   ├─ status (new/acknowledged/resolved)
   ├─ timestamp
   └─ location_data (lat, long, camera_id)

3. analytics_cache
   ├─ metric_name (string)
   ├─ value (number/JSON)
   ├─ time_bucket (hourly/daily)
   └─ last_updated
```

API Endpoints:

```
WebSocket:
  /ws
    - Bidirectional real-time communication
    - Events:
```

```
    • process_frame (receive from Testing Dashboard)
    • annotated_frame (send to Testing Dashboard)
    • new_alert (broadcast to Admin Panel)

REST API:
  POST /api/upload
    - Alternative to WebSocket for single image

  GET /api/alerts
    - Query parameters: severity, type, start_date, end_date
    - Returns: List of alerts

  GET /api/analytics/summary
    - Returns: Overall statistics
    - Response: {
        total_detections,
        by_type {pothole, animal, accident},
        by_severity {low, medium, high},
        today_count,
        hourly_breakdown
      }

  GET /api/analytics/heatmap
    - Returns: Location-based frequency data

  GET /api/detections/:id
    - Get specific detection details

  PATCH /api/alerts/:id
    - Update alert status (acknowledge/resolve)
```

2.3 MAIN ADMIN PANEL (Frontend 2)

Technology Stack:

- React.js / Next.js
- WebSocket (Socket.io client)
- Chart.js / Recharts (for analytics)
- Leaflet / Mapbox (for heatmap)
- Tailwind CSS / Material-UI

Purpose:

Command center for traffic authorities to monitor alerts and view analytics

Features:

A. Real-Time Alert Feed

- **Alert Cards**: Display new alerts as they arrive
  - Type icon (pothole/animal/accident)
  - Severity badge (color-coded)
  - Timestamp
  - Location (Camera ID)
  - Thumbnail image
  - Action buttons (Acknowledge, View Details)
- **Sorting/Filtering**:
  - By severity (High → Low)
  - By type
  - By time (Latest first)
  - By status (New/Acknowledged/Resolved)

B. Analytics Dashboard

Metric Cards:

- Total Detections (today)
- Active Alerts (unresolved)
- Potholes Detected
- Animals Detected
- Accidents Detected
- Average Response Time

Charts:

- **Line Chart**: Incidents over time (hourly/daily)
- **Bar Chart**: Detections by type
- **Pie Chart**: Severity distribution
- **Area Chart**: Traffic disruption trends

Heatmap:

- **Map View**: Shows camera locations
- **Color Intensity**: Frequency of detections
- **Hotspot Markers**: Areas with most animal activity
- **Click Details**: View location-specific history

C. Historical Log

- **Table View**: All past detections
- **Search**: By date, type, severity
- **Export**: Download as CSV/PDF
- **Details Modal**: Click to see full detection info

D. Settings Panel (Optional)

- Alert threshold configuration

- Camera management
- User preferences

Data Flow (Admin Panel):

- **Real-time (WebSocket)**: New alert notifications
- **Periodic (REST API)**: Analytics data (every 30 seconds)
- **On-demand (REST API)**: Historical queries, filters

---

3. COMPLETE DATA FLOW

3.1 PRIMARY FLOW (Real-Time Detection)

```
┌──────────────
```

```
│        frame_resized = cv2.resize(frame, (640, 480))      │
│                                                            │
│  ┌──────────────────────────────────────────────────────┐  │
│  │  Step 9: AI MODEL INFERENCE          │               │
│  │  (Parallel Execution)                │               │
│  │                                      │               │
│  │  Thread 1: Pothole YOLO              │               │
│  │  └─ Result: [bbox: [x,y,w,h], conf]  │               │
│  │                                      │               │
│  │  Thread 2: Animal YOLO               │               │
│  │  └─ Result: [bbox, class, conf]      │               │
│  │                                      │               │
│  │  Thread 3: Accident Detector         │               │
│  │  └─ Result: {is_accident, severity}  │               │
│  └──────────────────────────────────────────────────────┘  │
│                    ↓                                       │
│  Step 10: Feature Extraction                               │
│        For each detection:                                 │
│        ├─ Calculate severity (pothole size analysis)       │
│        ├─ Classify behavior (animal pose estimation)       │
│        ├─ Assess risk (multi-factor scoring)               │
│        └─ Apply context (road position, traffic)           │
│                    ↓                                       │
│  Step 11: Multi-Hazard Fusion                              │
│        Combine risks across modules                        │
│        Example: Animal + Pothole + Fast Vehicle            │
│              = CRITICAL combined risk                      │
│                    ↓                                       │
│  Step 12: Visual Annotation                                │
│        annotated_frame = frame.copy()                      │
│        For each detection:                                 │
│        ├─ cv2.rectangle() - Draw box                       │
│        ├─ cv2.putText() - Add label                        │
│        └─ Color by severity (Red/Orange/Green)             │
│                    ↓                                       │
│  Step 13: Alert Decision                                   │
│        IF severity >= 70 OR risk = CRITICAL:               │
│        ├─ Create alert object                              │
│        ├─ Save to database                                 │
│        └─ Queue for Admin Panel                            │
│                    ↓                                       │
│  Step 14: Encode output                                    │
│        annotated_base64 = encode(annotated_frame)          │
│                    ↓                                       │
│  Step 15: Send response via WebSocket                      │
│        socket.emit('annotated_frame', {                    │
│          annotatedFrame: base64,                           │
│          detections: [...],                                │
```

```
│            timestamp: 1.5                          │
│          })                                        │
└────────────────────────────────────────────────────┘
```

```
                  ↓
              (Network: 15ms)
                  ↓
┌──────────────────────────────────────────────────────────────────────────┐
│            TESTING DASHBOARD (Browser)                                   │
│                                                                          │
│  Step 16: Receive annotated frame                                        │
│  Step 17: Display frame                                                  │
│          <img src={data.annotatedFrame} />                               │
│          → User sees video with bounding boxes                           │
│                                                                          │
│  Step 18: Update detection sidebar                                       │
│          Map detections to UI cards                                      │
│          Show: Type, Severity, Confidence                                │
│                                                                          │
│  Step 19: Check for critical alerts                                      │
│          IF detections include high severity:                            │
│          └─ Trigger sound/visual notification                            │
│                                                                          │
│  Step 20: Extract next frame                                             │
│          → Loop back to Step 3                                           │
└──────────────────────────────────────────────────────────────────────────┘
```

3.2 SECONDARY FLOW (Alert to Admin Panel)

```
┌──────────────────────────────────────────────────────────────────────────┐
│            FASTAPI BACKEND                                               │
│  Critical detection identified (Step 13 above)                           │
└──────────────────────────────────────────────────────────────────────────┘
                  ↓
┌──────────────────────────────────────────────────────────────────────────┐
│          ALERT PROCESSING                                                │
│                                                                          │
│  Step A: Create Alert Record                                             │
│         alert = {                                                        │
│           id: UUID,                                                      │
│           type: "pothole",                                               │
│           severity: "high",                                              │
│           severity_score: 85,                                            │
```

```
│            timestamp: ISO datetime,                          │
│            location: "Camera-5",                             │
│            camera_coords: [lat, lng],                        │
│            detection_data: {...},                            │
│            image_thumbnail: base64_crop,                     │
│            status: "new"                                     │
│          }                                                   │
│                      ↓                                       │
│   Step B: Save to Database                                   │
│          db.alerts.insert_one(alert)                         │
│          db.detections.insert_one(detection)                 │
│                      ↓                                       │
│   Step C: Broadcast via WebSocket                            │
│          admin_socket.emit('new_alert', alert)               │
│                      ↓                                       │
│   Step D: Update Analytics Cache                             │
│          Increment counters:                                 │
│          ├─ total_alerts_today += 1                          │
│          ├─ pothole_count += 1                               │
│          └─ high_severity_count += 1                         │
└──────────────────────────────────────────────────────────────┘
                      ↓
┌──────────────────────────────────────────────────────────────┐
│          MAIN ADMIN PANEL (Browser)                          │
│                                                              │
│   Step E: Receive Alert (WebSocket)                          │
│          socket.on('new_alert', (alert) => {...})            │
│                      ↓                                       │
│   Step F: Update UI                                          │
│          ├─ Add alert card to feed (top)                     │
│          ├─ Show browser notification                        │
│          ├─ Play alert sound                                 │
│          └─ Increment metric counters                        │
│                      ↓                                       │
│   Step G: User Action (Optional)                             │
│          Authority clicks "Acknowledge"                      │
│          → PATCH /api/alerts/:id {status: "acknowledged"}    │
│          → Backend updates database                          │
│          → Status badge changes in UI                        │
└──────────────────────────────────────────────────────────────┘
```

3.3 ANALYTICS FLOW

```
┌──────────────────────────────────────────────────────────────┐
```

```
│          MAIN ADMIN PANEL (Browser)                    │
│  Page loads / Every 30 seconds                         │
└────────────────────────────────────────────────────────┘
                          ↓
                 GET /api/analytics/summary
                          ↓
┌────────────────────────────────────────────────────────┐
│            FASTAPI BACKEND                             │
│                                                        │
│  Query 1: Total Detections                             │
│          SELECT COUNT(*) FROM detections               │
│                                                        │
│  Query 2: Breakdown by Type                            │
│          SELECT type, COUNT(*) FROM detections         │
│          GROUP BY type                                 │
│                                                        │
│  Query 3: Breakdown by Severity                        │
│          SELECT severity, COUNT(*) FROM alerts         │
│          GROUP BY severity                             │
│                                                        │
│  Query 4: Today's Count                                │
│          SELECT COUNT(*) FROM detections               │
│          WHERE timestamp >= TODAY                      │
│                                                        │
│  Query 5: Hourly Breakdown                             │
│          SELECT HOUR(timestamp), COUNT(*)              │
│          FROM detections                               │
│          WHERE timestamp >= TODAY                      │
│          GROUP BY HOUR(timestamp)                      │
│                                                        │
│  Aggregate Results into JSON                           │
│          {                                             │
│            total_detections: 1247,                     │
│            by_type: {pothole: 523, animal: 412, ...},  │
│            by_severity: {high: 89, medium: 234, ...},  │
│            today: 47,                                  │
│            hourly: [{hour: 0, count: 3}, ...]          │
│          }                                             │
└────────────────────────────────────────────────────────┘
                          ↓
                    Return JSON
                          ↓
┌────────────────────────────────────────────────────────┐
│          MAIN ADMIN PANEL (Browser)                    │
```

```
│                                                │
│  Receive analytics data                        │
│  Update UI components:                         │
│  ├─ Metric cards (numbers)                     │
│  ├─ Line chart (hourly data)                   │
│  ├─ Bar chart (by type)                        │
│  └─ Pie chart (severity distribution)          │
└────────────────────────────────────────────────┘
```

---

4. QUICK REFERENCE Q&A

Q1: Does it send whole video or single frames?

A: Single frames, one at a time. Each frame is ~80KB.

Q2: Does backend return video or data?

A: Both - An annotated image (with boxes already drawn) + JSON detection data.

Q3: Does it send each frame or collectively?

A: Each frame individually in real-time loop.

Q4: Are boxes drawn in backend or frontend?

A: Backend draws boxes using OpenCV (recommended). Frontend just displays the image.

Q5: How fast is it?

A: 100-200ms per frame = 5-10 FPS output (smooth for demo).

Q6: Why not use Spring Boot?

A: Python FastAPI is faster to build, natively supports ML libraries, and handles WebSockets easily.

Q7: Where is the database?

A: Connected to FastAPI backend. Stores all detections, alerts, and analytics.

Q8: Can we demo without internet?

A: Yes - Run everything locally (localhost). No external dependencies needed.

Q9: What if models are slow?

A: Skip frames (process every 3rd-5th frame), resize images to 640x480, use YOLOv8-nano.

Q10: How to showcase all features in demo?

A: Prepare 3 test videos:

- Video 1: Multiple potholes (shows severity classification)
- Video 2: Stray animals crossing (shows behavior detection)
- Video 3: Near-miss accident (shows accident detection + severity scoring)

---

5. IMPLEMENTATION PRIORITY (24-Hour Timeline)

HOUR 0-6: CORE FOUNDATION (MUST HAVE)

Backend:

- ✅ FastAPI setup with WebSocket
- ✅ Basic frame processing pipeline
- ✅ Load pre-trained YOLO models (pothole, animal)
- ✅ Simple bounding box drawing

Testing Dashboard:

- ✅ Video upload interface
- ✅ WebSocket connection
- ✅ Display annotated frames

Goal: End-to-end flow working (even if basic)

HOUR 6-12: AI FEATURES (CORE USPs)

Backend:

- ✅ Pothole severity classification
- ✅ Animal behavior detection (basic)
- ✅ Accident detection (motion-based)
- ✅ Database integration

Testing Dashboard:

- ✅ Detection sidebar with live updates
- ✅ Severity badges/colors

Admin Panel:

- ✅ Basic alert feed
- ✅ WebSocket connection for alerts

Goal: All 3 detection modules working + alerts flowing

HOUR 12-18: POLISH & ANALYTICS (IMPRESSIVE)

Backend:

- ✅ Multi-hazard fusion logic
- ✅ Analytics endpoints
- ✅ Alert generation rules

Admin Panel:

- ✅ Analytics dashboard
- ✅ Charts (line, bar, pie)
- ✅ Metric cards

Testing Dashboard:

- ✅ Better UI/UX
- ✅ Loading states

Goal: Complete system with analytics

HOUR 18-22: ADVANCED FEATURES (BONUS)

If time permits:

- ✅ Heatmap visualization
- ✅ Night-time detection demo
- ✅ Hit-and-run detection logic
- ✅ Trajectory prediction (animals)

Goal: Showcase unique USPs

HOUR 22-24: TESTING & PRESENTATION

Tasks:

- ✅ End-to-end testing
- ✅ Prepare demo videos (pre-tested)
- ✅ Create presentation slides
- ✅ Setup backup (if live demo fails)
- ✅ Document setup instructions

---

6. DEPLOYMENT ARCHITECTURE

Local Development (Recommended for Hackathon)

```
Your Laptop:
├─ Port 8000: FastAPI Backend
├─ Port 3000: Testing Dashboard
├─ Port 3001: Admin Panel
└─ Port 27017: MongoDB (or PostgreSQL 5432)

Network:
└─ All communicate via localhost (no internet needed)
```

For Remote Judging (Optional)

```
Backend:
├─ Deploy to: Render / Railway / Heroku
└─ URL: https://your-backend.onrender.com

Frontends:
├─ Deploy to: Vercel / Netlify
```

```
├─ Testing Dashboard: https://testing.vercel.app
└─ Admin Panel: https://admin.vercel.app

Database:
└─ MongoDB Atlas (free tier) / Supabase
```

---

7. TECHNOLOGY JUSTIFICATION

Component	Choice	Why
Backend Framework	FastAPI	Fast, async, WebSocket support, Python-native
AI Models	YOLOv8	Pre-trained, fast inference, easy fine-tuning
Frontend	React	Component reusability, fast development
WebSocket	Socket.io	Real-time bidirectional, easy to use
Database	MongoDB	Flexible schema, fast writes, easy setup
Image Processing	OpenCV	Industry standard, Python integration
Charts	Chart.js	Simple, lightweight, good visuals

---

8. FILE STRUCTURE

```
project-root/
│
├── backend/                # FastAPI Backend
│   ├── main.py             # FastAPI app entry point
│   ├── models/             # AI model files
│   │   ├── pothole.pt
│   │   ├── animal.pt
│   │   └── vehicle.pt
│   ├── services/
│   │   ├── ai_engine.py       # AI inference logic
│   │   ├── feature_extraction.py
│   │   └── alert_manager.py
```

```
│   ├── api/
│   │   ├── websocket.py      # WebSocket handlers
│   │   └── routes.py         # REST API endpoints
│   ├── database/
│   │   ├── models.py         # DB schemas
│   │   └── connection.py
│   └── requirements.txt
│
├── testing-dashboard/        # React Frontend 1
│   ├── src/
│   │   ├── components/
│   │   │   ├── VideoUploader.jsx
│   │   │   ├── VideoPlayer.jsx
│   │   │   └── DetectionSidebar.jsx
│   │   ├── services/
│   │   │   └── websocket.js
│   │   └── App.jsx
│   └── package.json
│
├── admin-panel/              # React Frontend 2
│   ├── src/
│   │   ├── components/
│   │   │   ├── AlertFeed.jsx
│   │   │   ├── Analytics.jsx
│   │   │   └── Heatmap.jsx
│   │   ├── services/
│   │   │   ├── api.js
│   │   │   └── websocket.js
│   │   └── App.jsx
│   └── package.json
│
└── README.md
```

---

9. SUCCESS METRICS (What Judges Will See)

Demo Flow:

1. **Upload test video** → System processes in real-time
2. **Bounding boxes appear** → Shows AI is working
3. **Alerts pop up in Admin Panel** → Shows integration
4. **Analytics update** → Shows data intelligence

Wow Factors:

- ✅ Real-time processing feels instant
- ✅ Color-coded severity (visual impact)
- ✅ Multiple hazard types detected simultaneously
- ✅ Professional dashboard UI
- ✅ Live analytics charts

Technical Highlights to Mention:

- ✅ Multi-modal AI (3 detection types)
- ✅ Behavior-based intelligence (not just object detection)
- ✅ Scalable architecture (ready for real CCTVs)
- ✅ Context-aware risk assessment (fusion logic)

---

10. RISK MITIGATION

Backup Plans:

If AI is too slow:

- Use smaller models (YOLOv8-nano)
- Process fewer frames (every 5th frame)
- Pre-process demo videos

If WebSocket fails:

- Fall back to REST API polling
- Use pre-recorded results

If database is down:

- Use in-memory storage (Python dict)
- Show cached analytics

If live demo crashes:

- Have recorded demo video ready
- Explain architecture from slides